Practical 12

Implementation of Travelling Salesman Problem (TSP)

1. Introduction

The Travelling Salesman Problem (TSP) is a classic optimization problem in computer science and algorithm design. A salesman is given a set of cities and the cost (distance) of travelling between every pair of cities. The salesman must start from a given city, visit every other city exactly once, and return to the starting city, such that the total travelling cost is minimum.

In this implementation, the set of visited cities is stored using a **bitmask**: bit i of the mask is 1 if city i has already been visited. The function tries every unvisited city as the next city recursively and keeps the minimum cost of completing the tour.

2. Aim

To implement the Travelling Salesman Problem in C++ using a bitmask-based recursive approach and determine the minimum cost of a tour that visits every city exactly once and returns to the starting city.

3. Objectives

- To understand the concept of the Travelling Salesman Problem.
- To understand the use of a bitmask to represent the set of visited cities.
- To find the minimum cost tour through all cities.
- To understand the recursive exploration of all possible tours.
- To analyze the time and space complexity of the algorithm.
- To implement the algorithm using C++.

4. Algorithm

Step 1: Start.

Step 2: Read the number of cities N and the cost matrix cost[N][N], where cost[i][j] is the cost of travelling from city i to city j.

Step 3: Start from city 0 with visited = 1 (only city 0 marked as visited).

Step 4: If all cities are visited (visited = $2^N - 1$), return the cost of going back from the current city to city 0.

Step 5: Set ans = infinity.

Step 6: For each city i from 0 to N–1, if city i is not visited:

- Mark city i as visited and recursively compute the cost of the remaining tour.

- newCost = cost[city][i] + tsp(i, visited | (1 << i)).
- If newCost < ans, update ans = newCost.

Step 7: Return ans as the minimum cost from the current state.

Step 8: The value returned by tsp(0, 1) is the minimum travelling cost.

Step 9: Print the result.

Step 10: Stop.

5. Pseudocode

```
BEGIN
FUNCTION TSP(cost, city, visited)
   IF visited == (1 << N) - 1 THEN
      RETURN cost[city][0]
   END IF

   ans = INFINITY

   FOR i = 0 TO N-1
      IF city i is not visited THEN
         newCost = cost[city][i] + TSP(cost, i, visited | (1 << i))
         IF newCost < ans THEN
            ans = newCost
         END IF
      END IF
   END FOR

   RETURN ans
END FUNCTION

MAIN: PRINT TSP(cost, 0, 1)
END
```

Code:

```
#include <iostream>
#include <climits>
using namespace std;

#define N 4

int tsp(int cost[N][N], int city, int visited) {
    if (visited == (1 << N) - 1)
        return cost[city][0];
```

```
    int ans = INT_MAX;

    for (int i = 0; i < N; i++) {
        if (!(visited & (1 << i))) {
            int newCost = cost[city][i] +
                tsp(cost, i, visited | (1 << i));

            if (newCost < ans)
                ans = newCost;
        }
    }

    return ans;
}

int main() {
    int cost[N][N] = {
        {0, 10, 15, 20},
        {10, 0, 35, 25},
        {15, 35, 0, 30},
        {20, 25, 30, 0}
    };

    cout << "Minimum travelling cost: "
         << tsp(cost, 0, 1);

    return 0;
}
```

Input Cost Matrix

The program uses 4 cities (0 to 3) with the following cost matrix. The tour starts and ends at city 0.

From \ To	0	1	2	3
0	0	10	15	20
1	10	0	35	25
2	15	35	0	30
3	20	25	30	0

Output

```
Minimum travelling cost: 80
```

Explanation: starting from city 0, the algorithm explores all 3! = 6 possible tours:

Tour	Cost Calculation	Total
$0 \to 1 \to 2 \to 3 \to 0$	10 + 35 + 30 + 20	95
$0 \to 1 \to 3 \to 2 \to 0$	10 + 25 + 30 + 15	80
$0 \to 2 \to 1 \to 3 \to 0$	15 + 35 + 25 + 20	95
$0 \to 2 \to 3 \to 1 \to 0$	15 + 30 + 25 + 10	80
$0 \to 3 \to 1 \to 2 \to 0$	20 + 25 + 35 + 15	95
$0 \to 3 \to 2 \to 1 \to 0$	20 + 30 + 35 + 10	95

The minimum cost is **80**, achieved by the tours 0→1→3→2→0 and 0→2→3→1→0 (the same tour in opposite directions, since the matrix is symmetric).

6. Advantages

- Always finds the optimal (minimum cost) tour.
- Simple to understand and implement using recursion and bitmasking.
- The bitmask gives a compact and fast way to track visited cities.
- Works for any cost matrix, including asymmetric costs.
- Suitable for small numbers of cities.

7. Disadvantages

- Explores every possible tour, so it becomes very slow as the number of cities grows.
- This version does not store (memoize) results, so the same subproblems are recomputed.
- Not practical for large inputs; heuristic or approximation methods are used instead.
- TSP is NP-hard, so no known algorithm solves it in polynomial time.
- The cost matrix must be complete (every city reachable from every other city).

8. Complexity Analysis

Complexity	**Value**
Time Complexity	O(N!)
Space Complexity	O(N)

Where N = number of cities. Space is used only by the recursion stack (depth N). If the results of each state (city, visited) are stored in a DP table, the time reduces to $O(N^2 \times 2^N)$ at the cost of $O(N \times 2^N)$ space (Held–Karp algorithm).

9. Conclusion

The Travelling Salesman Problem was successfully implemented in C++ using a bitmask-based recursive approach. The algorithm tries every possible order of visiting the cities and keeps the minimum total cost. For the given 4-city cost matrix, the program produced the minimum travelling cost of 80. This approach guarantees the optimal answer for small inputs, and it can be made more efficient by adding memoization (Held–Karp dynamic programming).